

Iterative Methods for Linear Equations

CL249 11-08-2026

Iterative Methods for Linear Systems

Gauss-Jacobi, Gauss-Seidel, and Comparison with Gaussian Elimination / LU

Applied Numerical Methods & Computational Science Reference

ITERATIVE VS. DIRECT SOLVERS

Introduction to Iterative Methods

Mathematical Framework

- Objective: Solve large linear systems $Ax = b$ by starting from an initial guess $x^{(0)}$ and generating successive approximations $x^{(1)}, x^{(2)}, \dots$
- Stationary Form: Matrix A is split into $A = D + L + U$ (Diagonal, Strictly Lower, Strictly Upper), converting $Ax = b$ to $x^{(k+1)} = M x^{(k)} + c$.
- Stopping Criterion: Iterations continue until the relative residual norm drops below tolerance: $\|b - A x^{(k)}\| / \|b\| < \text{tol}$.
- Target Applications: Essential for ultra-large, sparse systems (e.g., Finite Element Analysis, Discretized Partial Differential Equations).

Gauss-Jacobi Iteration Method

Synchronous Update Mechanism

- Update Formula: $x_i^{(k+1)} = \left(\frac{1}{a_{ii}}\right) \left[b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}\right]$
- Synchronous Updates: ALL elements of vector $x^{(k+1)}$ are computed simultaneously using strictly values from the previous iteration step k.
- Iteration Matrix: $M_J = -D^{-1}(L + U)$, where D is the diagonal matrix of A.
- Memory Requirement: Requires holding two full vectors in memory at once ($x^{(k)}$ and $x^{(k+1)}$).
- Parallel Computing Advantage: Fully decoupled computations make Jacobi embarrassingly parallelizable on modern GPU architectures.

Gauss-Seidel Iteration Method

Sequential In-Place Refinement

- Update Formula: $x_i^{(k+1)} = \left(\frac{1}{a_{ii}}\right) * [b_i - \sum_{j < i}^n (a_{ij} \cdot x_j^{(k+1)}) - \sum_{j > i} (a_{ij} \cdot x_j^k)]$
- In-Place Updates: Newly computed values x_i^{k+1} for $j < i$ immediately overwrite old ones within the same iteration pass.
- Iteration Matrix: $M_{GS} = -(D + L)^{-1} U$.
- Faster Convergence: Due to immediate feedback, Gauss-Seidel typically converges roughly TWICE as fast as Gauss-Jacobi ($\rho(M_{GS}) \approx \rho(M_J)^2$).
- Reduced Memory Footprint: Requires storing only a single state vector in memory during iteration.

Diagonal Dominance & Matrix Decomposition

Strict Diagonal Dominance

Definition: A matrix A is strictly diagonally dominant if for every row i:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

Significance: If A is strictly diagonally dominant, both Jacobi and Gauss-Seidel are GUARANTEED to converge for ANY initial guess $x^{(0)}$.

Row Swapping Strategy: Always reorder system equations if needed to ensure maximum pivot values appear on the diagonal!

Matrix Splitting $A = D + L + U$

Any matrix A is decomposed into three components:

- D : Diagonal matrix containing diagonal entries a_{ii}
- L : Strictly lower triangular matrix
- U : Strictly upper triangular matrix

System Form: $(D + L + U)x = b$

Goal: Rearrange into stationary iterative form:

$$x^{(k+1)} = M x^{(k)} + c$$

where M is the characteristic iteration matrix.

Derivation of M_Jacobi and M_Gauss-Seidel

1. Jacobi Iteration Matrix (M_J):

$$\text{Split: } Dx^{(k+1)} = b - (L + U)x^{(k)} \Rightarrow x^{(k+1)} = -D^{-1}(L + U)x^k + D^{-1}b$$

$$\text{Iteration Matrix: } M_J = -D^{-1}(L + U)$$

2. Gauss-Seidel Iteration Matrix (M_{GS}):

$$\text{Split: } (D + L)x^{(k+1)} = b - Ux^{(k)} \Rightarrow x^{(k+1)} = -(D + L)^{-1}Ux^k + (D + L)^{-1}b$$

$$\text{Iteration Matrix: } M_{GS} = -(D + L)^{-1}U$$

3. Role of M Matrix in Convergence:

Error propagation satisfies $e^k = M^k e^{(0)}$.

The iteration converges if and only if spectral radius

$$\rho(M) = \max|\lambda_i| < 1.$$

Smaller $\rho(M) \Rightarrow$ Faster error reduction per step!

Worked Example: System Definition & Diagonal Dominance Test

Benchmark 3x3 Linear System

Consider the linear system $Ax = b$:

$$10x_1 + x_2 + 2x_3 = 13$$

$$x_1 + 10x_2 + x_3 = 12$$

$$2x_1 + 3x_2 + 10x_3 = 15$$

Exact Solution: $x^* = [1, 1, 1]^T$

Initial Guess: $x^{(0)} = [0, 0, 0]^T$

Checking Strict Diagonal Dominance:

- Row 1: $|10| = 10 > |1| + |2| = 3$ ✓ (Satisfied)
- Row 2: $|10| = 10 > |1| + |1| = 2$ ✓ (Satisfied)
- Row 3: $|10| = 10 > |2| + |3| = 5$ ✓ (Satisfied)

Result: Strict diagonal dominance holds! Both Jacobi and Gauss-Seidel are guaranteed to converge.

Worked Example: Gauss-Jacobi Step-by-Step

Synchronous Iteration Formulas & Calculation

Explicit Equations for $x^{(k+1)}$:

$$x_1^{(k+1)} = (13 - x_2^{(k)} - 2 x_3^{(k)}) / 10$$

$$x_2^{(k+1)} = (12 - x_1^{(k)} - x_3^{(k)}) / 10$$

$$x_3^{(k+1)} = (15 - 2 x_1^{(k)} - 3 x_2^{(k)}) / 10$$

Iteration 1 (from $x^{(0)} = [0, 0, 0]^T$):

- $x_1^{(1)} = (13 - 0 - 0) / 10 = 1.300$
- $x_2^{(1)} = (12 - 0 - 0) / 10 = 1.200$
- $x_3^{(1)} = (15 - 0 - 0) / 10 = 1.500 \Rightarrow x^{(1)} = [1.300, 1.200, 1.500]^T$

Iteration 2 (using ONLY $x^{(1)}$ values):

- $x_1^{(2)} = (13 - 1.2 - 2(1.5)) / 10 = (13 - 4.2) / 10 = 0.880$
- $x_2^{(2)} = (12 - 1.3 - 1.5) / 10 = (12 - 2.8) / 10 = 0.920$
- $x_3^{(2)} = (15 - 2(1.3) - 3(1.2)) / 10 = (15 - 6.2) / 10 = 0.880 \Rightarrow x^{(2)} = [0.880, 0.920, 0.880]^T$

Worked Example: Gauss-Seidel Step-by-Step

In-Place Sequential Update Computation

Explicit Equations for $x^{(k+1)}$ (Using Updated Values Immediately):

$$x_1^{(k+1)} = (13 - x_2^{(k)} - 2 x_3^{(k)}) / 10$$

$$x_2^{(k+1)} = (12 - x_1^{(k+1)} - x_3^{(k)}) / 10 \quad \leftarrow \text{Uses new } x_1^{(k+1)}$$

$$x_3^{(k+1)} = (15 - 2 x_1^{(k+1)} - 3 x_2^{(k+1)}) / 10 \quad \leftarrow \text{Uses new } x_1^{(k+1)} \text{ and } x_2^{(k+1)}$$

Iteration 1 (from $x^{(0)} = [0, 0, 0]^T$):

- $x_1^{(1)} = (13 - 0 - 0) / 10 = 1.3000$

- $x_2^{(1)} = (12 - 1.3000 - 0) / 10 = 1.0700 \quad (\text{Uses } x_1^{(1)} = 1.3000)$

- $x_3^{(1)} = (15 - 2(1.3000) - 3(1.0700)) / 10 = (15 - 5.81) / 10 = 0.9190 \Rightarrow x^{(1)} = [1.3000, 1.0700, 0.9190]^T$

Iteration 2 (In-place updates):

- $x_1^{(2)} = (13 - 1.0700 - 2(0.9190)) / 10 = (13 - 2.9080) / 10 = 1.0092$

- $x_2^{(2)} = (12 - 1.0092 - 0.9190) / 10 = (12 - 1.9282) / 10 = 1.0072$

- $x_3^{(2)} = (15 - 2(1.0092) - 3(1.0072)) / 10 = (15 - 5.0400) / 10 = 0.9960 \Rightarrow x^{(2)} = [1.0092, 1.0072, 0.9960]^T$

Convergence Trajectory & Summary Comparison

Iteration k	Exact Solution x^*	Gauss-Jacobi $x^{(k)}$	Gauss-Seidel $x^{(k)}$
$k = 0$	[1.000, 1.000, 1.000]	[0.000, 0.000, 0.000]	[0.0000, 0.0000, 0.0000]
$k = 1$	[1.000, 1.000, 1.000]	[1.300, 1.200, 1.500]	[1.3000, 1.0700, 0.9190]
$k = 2$	[1.000, 1.000, 1.000]	[0.880, 0.920, 0.880]	[1.0092, 1.0072, 0.9960]

Key Takeaways from the Numerical Example

- Precision After Step 2: Gauss-Seidel is already within $\sim 0.9\%$ of the exact solution $[1, 1, 1]$, whereas Jacobi still suffers $\sim 12\%$ error.
- Why GS Converges Faster: In-place substitution immediately incorporates newly available estimates (e.g. $x_1^{(1)} = 1.3000$ into row 2), accelerating the decay of error components.
- Spectral Radius Connection: For this system, $\rho(M_{GS}) \approx 0.025$ while $\rho(M_J) \approx 0.158$. Since $\rho(M_{GS}) \ll \rho(M_J)$, Gauss-Seidel requires far fewer iterations to reach tolerance.

Direct Methods Refresher: Gaussian Elimination & LU

Exact Solvers & Matrix Factorization

- Gaussian Elimination: Reduces $[A \mid b]$ to upper triangular form $[U \mid y]$ via row operations, followed by back-substitution to solve for x .
- LU Factorization ($PA = LU$): Decomposes matrix A into lower (L) and upper (U) triangular matrices with permutation matrix P .
- Deterministic Solution: Guarantees exact solution in a fixed finite number of operations (absent machine round-off errors).
- Multiple RHS Efficiency: Once LU is computed in $O(N^3)$ operations, solving $Ax_i = b_i$ for multiple right-hand sides takes only $O(N^2)$ per vector.
- Main Limitation: Fails to preserve sparsity — causes 'fill-in', dramatically inflating RAM and compute requirements for huge sparse matrices.

Understanding Spectral Radius: Concept & Convergence

Definition & Mathematical Core

Definition:

The spectral radius $\rho(M)$ of an $n \times n$ matrix M is the maximum absolute value among all its eigenvalues (λ):

$$\rho(M) = \max_i |\lambda_i|$$

Connection to Iterative Methods:

- An iterative process $x^{(k+1)} = M x^{(k)} + c$ has error vector $e^{(k)} = x^{(k)} - x^*$.
- Error propagation is governed by matrix powers: $e^{(k)} = M^k e^{(0)}$.
- As $k \rightarrow \infty$, $M^k \rightarrow 0$ if and only if $\rho(M) < 1$.

Why Spectral Radius Governs Speed

Convergence Condition:

$\rho(M) < 1 \Rightarrow$ Method Converges!

$\rho(M) \geq 1 \Rightarrow$ Method Diverges or Stalls.

Asymptotic Rate of Convergence:

- The error reduction factor per iteration is roughly bounded by $\rho(M)$:
$$\|e^{(k+1)}\| \approx \rho(M) \cdot \|e^{(k)}\|$$
- Smaller $\rho(M)$ = Faster decay of error.
- Example: If $\rho(M) = 0.1$, error drops by 10x each step. If $\rho(M) = 0.9$, error drops by only 10% each step!

Please note down: For today's class, you need to calculate spectral radius

To find eigenvalues: you may use linear algebra package

$$np.linalg.eigvals(M)$$

Then, to find the maximum of the absolute values :

$$\max\left(\text{abs}(np.linalg.eigvals(M))\right)$$

Explicit Example: Calculating $\rho(M_{Jacobi})$ vs. $\rho(M_{GS})$ — 2x2 Linear System Example Analysis

Consider system $A = \begin{bmatrix} 4 & 2 \\ 1 & 4 \end{bmatrix}$, $b = \begin{bmatrix} 6 \\ 5 \end{bmatrix}^T$:

Split $A = D + L + U$ where $D = \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}$, $L = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$, $U = \begin{bmatrix} 0 & 2 \\ 0 & 0 \end{bmatrix}$

1. Jacobi Iteration Matrix $M_J = -D^{-1}(L + U)$:

$$M_J = - \begin{bmatrix} 1/4 & 0 \\ 0 & 1/4 \end{bmatrix} \begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -0.5 \\ -0.25 & 0 \end{bmatrix}$$

- Eigenvalues $\det(M_J - \lambda I) = 0 \Rightarrow \lambda^2 - 0.125 = 0 \Rightarrow \lambda = \pm\sqrt{0.125} \approx \pm 0.3536$
- Spectral Radius $\rho(M_J) = 0.3536$

2. Gauss-Seidel Iteration Matrix $M_{GS} = -(D + L)^{-1}U$:

$$(D + L)^{-1} = \begin{bmatrix} 1/4 & 0 \\ -1/16 & 1/4 \end{bmatrix} \Rightarrow M_{GS} = \begin{bmatrix} 0 & -0.5 \\ 0 & 0.125 \end{bmatrix}$$

- Eigenvalues $\det(M_{GS} - \lambda I) = \lambda(\lambda - 0.125) = 0 \Rightarrow \lambda_1 = 0, \lambda_2 = 0.125$
- Spectral Radius $\rho(M_{GS}) = 0.125$

Takeaway: $\rho(M_{GS}) = 0.125 = [\rho(M_J)]^2 = (0.3536)^2$. Gauss-Seidel reduces error almost 3x faster per iteration!

Direct Solvers vs. Iterative Methods

Feature	Direct Methods (Gauss / LU)	Iterative Methods (Jacobi / Seidel)
Solution Type	Exact (up to floating-point limits)	Approximate (converges within tolerance tol)
Time Complexity	Fixed $O(N^3)$ operations	$O(k \cdot N^2)$ operations for k iterations
Matrix Sparsity	Destroys sparsity (introduces fill-in)	Preserves matrix structure and zero patterns
Memory Footprint	High (requires $O(N^2)$ memory for L & U)	Low (stores non-zeros + minimal state vectors)
Error Resilience	Accumulates round-off errors continuously	Self-correcting / resilient to initial error

ITERATIVE VS. DIRECT SOLVERS

Advantages of Iterative Methods

Why Choose Iterative Solvers for Engineering & Physics?

1. Sparsity Preservation: Never generates zero fill-in, making it ideal for million-variable systems arising from PDE spatial discretizations.
2. Computational Scalability: When $N > 10^5$, direct $O(N^3)$ computation becomes impossible; iterative methods finish in $k \ll N$ passes.
3. Memory Economy: Avoids memory bottlenecks by storing matrix coefficients in Compressed Sparse Row (CSR) format.
4. Warm Starting & Early Termination: Can stop early when engineering precision is met, or start with a physical initial guess $x^{(0)}$ to converge rapidly.
5. High Parallelizability: *Jacobi and modern Krylov-subspace variants map cleanly onto GPU SIMD hardware.*